



Department of Computer Science and Engineering
DBMS Unit 2 Notes

UNIT 2 — Advanced SQL

Additional Basic Operations • Set Operations • Null Values • Aggregate Functions • Nested Subqueries • Database Modification • Join Expressions • Views • Triggers • Functions & Procedures • CTE • Window Functions • Full Text Search

Quick-Revision Notes

1. Additional Basic Operations

SELECT–FROM–WHERE structure

- **SELECT** clause lists desired attributes ($\approx$ relational algebra projection).
- **FROM** clause specifies the table(s)/relation(s) to query.
- **WHERE** clause filters rows using a condition ($\approx$ relational algebra selection).
- SQL keywords are case-insensitive: **SELECT** $\equiv$ **select** $\equiv$ **Select**.

```
SELECT Fname, Lname FROM EMPLOYEE;
```

DISTINCT clause

SQL allows duplicate rows by default. **DISTINCT** eliminates duplicate rows from the result.

```
SELECT DISTINCT Fname, Lname, Address FROM EMPLOYEE;
```

Arithmetic in SELECT & the AS clause

SELECT can contain expressions with +, -, *, / on constants/attributes. AS renames a column or relation (old-name AS new-name). The keyword AS is optional.

```
SELECT fname, salary/2 AS monthly_salary FROM EMPLOYEE;
```

WHERE clause

Specifies row-filtering conditions using logical connectives **AND**, **OR**, **NOT** and comparison operators <, <=, >, >=, =, <>.

```
SELECT fname, salary FROM employee  
WHERE dno = 5 AND salary < 70000;
```

ORDER BY clause

- Sorts the result in ascending (**ASC**, default) or descending (**DESC**) order.
- Can be combined with aggregate functions, GROUP BY or HAVING.

```
SELECT fname, lname, salary FROM EMPLOYEE ORDER BY salary DESC;
```

String comparison — LIKE

- % matches any substring; _ matches any single character.
- Use ESCAPE to match literal % or _, e.g. '100 \%' ESCAPE '\'.

```
SELECT Fname, Lname FROM EMPLOYEE  
WHERE Address LIKE '%Houston,TX%';  
  
-- employees born in the 1960s  
SELECT Fname,Lname,Bdate FROM EMPLOYEE  
WHERE Bdate LIKE '196_____';
```

2. Set Operations & Null Values

Set operators — combine two SELECT results

Operator	Behaviour
UNION	Combines rows from both queries; removes duplicates.
UNION ALL	Combines rows; keeps duplicates (count = c1 + c2).
INTERSECT	Common rows in both queries; no duplicates.
INTERSECT ALL	Common rows, duplicate count = min(c1, c2).
EXCEPT	Rows in query-1 but not query-2; no duplicates.
EXCEPT ALL	As EXCEPT, duplicate count = max(c1 - c2, 0).

Note: For UNION/INTERSECT/EXCEPT, both SELECTs must return the same number of columns with compatible types.

```
SELECT grade FROM student_course WHERE course='Physics'
UNION
SELECT grade FROM student_course WHERE course='Mathematics';
```

Null values — meanings

- **Unknown** — value exists but is not known (e.g. unknown DOB).
- **Unavailable/withheld** — value exists but is purposely hidden.
- **Not applicable** — attribute does not apply to this tuple (e.g. LastCollegeDegree for a non-graduate).
- SQL does not distinguish between these meanings; every NULL is treated as different from every other NULL.

NULL in arithmetic & comparisons

- Any arithmetic operation (+, -, *, /) with a NULL operand yields NULL.
- Comparisons with NULL evaluate to a third value: **UNKNOWN** (three-valued logic: TRUE / FALSE / UNKNOWN).
- e.g. (1 < NULL) → UNKNOWN; a tuple is kept only if WHERE evaluates to TRUE (not FALSE/UNKNOWN).

```
SELECT Fname, Lname FROM EMPLOYEE WHERE Super_ssn IS NULL; -- IS NULL / IS NOT NULL
SELECT Essn, Pno FROM WORKS_ON WHERE Hours > 10 IS UNKNOWN; -- IS UNKNOWN / IS NOT UNKNOWN
```

NULL in DISTINCT & set operations

For DISTINCT, UNION, INTERSECT, EXCEPT — two NULLs in corresponding positions are treated as *identical* (unlike in WHERE, where NULL = NULL gives UNKNOWN, not TRUE).

3. Aggregate Functions

Built-in aggregate functions

- **COUNT** — number of tuples/values. **SUM** — total of numeric values.
- **MAX / MIN** — largest / smallest value (works on any totally-ordered domain, e.g. DATE).

- **AVG** — average of numeric values.

```
SELECT SUM(Salary), MAX(Salary), MIN(Salary), AVG(Salary) FROM EMPLOYEE;
```

COUNT and DISTINCT

COUNT(*) counts result rows. COUNT(DISTINCT col) counts unique non-duplicate values in a column; without DISTINCT, duplicates are retained (SQL does not eliminate duplicates by default).

Handling of NULLs in aggregates

NULL values are discarded before an aggregate is computed. If the whole collection is NULL, COUNT returns 0, other aggregates return NULL.

GROUP BY clause

- Partitions a relation into groups (non-overlapping subsets) sharing the same value(s) of the **grouping attribute(s)**.
- A separate group is formed for NULL values of the grouping attribute.
- SELECT list may contain only grouping attributes and aggregate functions.

```
SELECT Dno, COUNT(*), AVG(Salary) FROM EMPLOYEE GROUP BY Dno;
```

HAVING clause

- WHERE filters individual tuples *before* grouping; HAVING filters whole groups *after* grouping/aggregation.
- Execution order: WHERE → GROUP BY → HAVING.

```
SELECT Pnumber, Pname, COUNT(*) FROM PROJECT, WORKS_ON
WHERE Pnumber = Pno GROUP BY Pnumber, Pname HAVING COUNT(*) > 2;
```

Note: To filter on salary AND on group-size simultaneously, nest a nested query inside WHERE instead of relying on HAVING alone — HAVING only sees the tuples that already survived WHERE.

Aggregate functions in nested (correlated) queries

```
SELECT Lname, Fname FROM EMPLOYEE
WHERE (SELECT COUNT(*) FROM DEPENDENT WHERE Ssn = Essn) >= 2;
```

SOME / ALL Boolean functions: **SOME** is True if ≥ 1 element is True; **ALL** is True if every element is True.

4. Join Expressions

A join combines two relations (Cartesian product + matching condition) into one result relation. Join expressions are typically used as subquery expressions in the FROM clause.

Natural Join

- Implicit EQUIJOIN on all attribute-pairs sharing the same name & type; each such pair appears once.
- No ON clause is used; rename columns first if matching names differ.

```
SELECT employee_name, employee_salary, Department_name
FROM employee NATURAL JOIN Department;
```

Inner Join (Equijoin)

Default join type — compares rows of both tables and keeps only matching pairs based on the join predicate.

```
SELECT e.Employee_ID, e.Employee_Name, d.Department_Name
FROM employee e INNER JOIN department d ON e.Employee_ID = d.Employee_ID;
```

Outer Joins

Type	Behaviour
LEFT OUTER JOIN	All rows from left table; unmatched right columns padded with NULL.
RIGHT OUTER JOIN	All rows from right table; unmatched left columns padded with NULL.
FULL OUTER JOIN	All rows from both tables (matched + unmatched, both sides).

Note: MySQL has no FULL OUTER JOIN — emulate with LEFT OUTER JOIN UNION RIGHT OUTER JOIN.

```
SELECT e.Employee_ID, d.Department_Name FROM employee e LEFT OUTER
JOIN department d ON e.Employee_ID = d.Employee_ID
UNION
SELECT e.Employee_ID, d.Department_Name FROM employee e RIGHT OUTER
JOIN department d
ON e.Employee_ID = d.Employee_ID;
```

Cross Join

Returns the plain Cartesian product — no join condition, no ON clause. Rows returned = (rows in A) × (rows in B).

```
SELECT * FROM loan CROSS JOIN borrower;
```

5. Nested Subqueries

Basics

- A nested query is a complete SELECT-FROM-WHERE block placed inside another query (the **outer query**).
- **IN / NOT IN** test set membership of a value in the result of a subquery.
- If a subquery returns a single value (single row, single column), it can be compared using =, <, > etc. instead of IN.

```
SELECT DISTINCT Essn FROM WORKS_ON
WHERE (Pno, Hours) IN
(SELECT Pno, Hours FROM WORKS_ON WHERE Essn = '123456789');
```

ANY / ALL / SOME comparison operators

Form: *comp_op* [ANY | ALL | SOME] (subquery). **ALL** — True if comparison holds for every element of the set.

SOME/ANY — True if it holds for at least one element. '= SOME' ≡ IN; '<> ALL' ≡ NOT IN.

```
SELECT Lname, Fname FROM EMPLOYEE
WHERE Salary > ALL (SELECT Salary FROM EMPLOYEE WHERE Dno = 5);
```

Correlated nested queries

The inner WHERE clause references an attribute of the outer query's relation, so the subquery is re-evaluated once per outer tuple.

```
SELECT E.Fname, E.Lname FROM EMPLOYEE AS E
WHERE E.Ssn IN
(SELECT D.Essn FROM DEPENDENT AS D WHERE E.Gender = D.Gender);
```

EXISTS / NOT EXISTS

- **EXISTS** — True if the nested query result is non-empty.
- **NOT EXISTS** — True if the nested query result is empty.
- Usually paired with a correlated nested query; commonly used for 'for-all' (universal) conditions via double-negation (NOT EXISTS ... EXCEPT ...).

```
-- employees with no dependents
SELECT Fname, Lname FROM EMPLOYEE
WHERE NOT EXISTS (SELECT * FROM DEPENDENT WHERE Ssn = Essn);
```

Subqueries in the FROM clause

A subquery can appear in FROM since it returns a relation. MySQL/PostgreSQL require the derived table to be given an alias name.

```
SELECT Dno, avg_salary FROM (SELECT Dno, ROUND(AVG(Salary),2) FROM EMPLOYEE GROUP BY Dno)
AS dept_avg_salary(Dno, avg_salary)
WHERE avg_salary > 32000;
```

Levels of nesting & naming ambiguity

An unqualified attribute refers to the relation in the *innermost* query that declares it. Use table aliases (tuple variables) to unambiguously reference outer-query attributes and to avoid errors.

6. Database Modification

The INSERT, DELETE and UPDATE statements modify the contents of the database; subqueries may be used inside all three to select the rows to act on.

INSERT

```
INSERT INTO EMPLOYEE (Fname, Lname, Ssn, Dno)
VALUES ('Ravi', 'Kumar', '999887776', 5);

-- insert rows produced by a query
INSERT INTO Marks_history
SELECT * FROM Marks_sample WHERE SRN = 2;
```

UPDATE

Modifies attribute values of tuples that satisfy a WHERE condition; the SET clause lists attribute = new-value pairs (may itself use a subquery).

```
UPDATE EMPLOYEE SET Salary = Salary * 1.1 WHERE Dno = 5;
```

DELETE

Removes whole tuples satisfying a WHERE condition (never deletes individual attribute values).

```
DELETE FROM Marks_sample WHERE SRN = 2;
```

Note: Without a WHERE clause, UPDATE/DELETE act on every row of the table — always double-check the condition.

7. Views, Users & Privileges

What is a view?

- A **virtual table** defined by a stored query; it does not store data itself.
- Hides query complexity, promotes reuse, and can restrict visible columns/rows for security.

```
CREATE VIEW view1 AS <query expression>;
```

Updatable views

A view supports CRUD operations only if **all** of the following hold:

- Defined on a single table only (no joins/sub-queries).
- No aggregate functions (SUM, COUNT, MAX...).
- No DISTINCT, GROUP BY, HAVING or UNION.
- Not recursive.

Note: Changes made through an updatable view are reflected in the underlying base table.

Views — advantages

- Reduce query complexity & improve query simplicity for end users.
- Increase security by hiding sensitive columns.
- Provide logical data independence and enforce data-integrity checks automatically.

Views vs CTE

Aspect	View	CTE
Persistence	Stored in schema until dropped	Exists only during query execution
Reusability	Reusable across many queries	Usable only within its own query
Scope	Global (permission based)	Local to the query
Updatable	Yes, if single-table & simple	No — read-only result set

Users, Roles & Privileges (MySQL)

```
CREATE USER 'student'@'localhost' IDENTIFIED BY 'stud123';
DROP USER 'student'@'localhost';

GRANT SELECT ON school.students TO 'alice'@'localhost';
GRANT ALL PRIVILEGES ON school.* TO 'bob'@'localhost';
SHOW GRANTS FOR 'bob'@'localhost'; REVOKE SELECT
ON school.* FROM 'alice'@'localhost';

CREATE ROLE Assistant_employee;
GRANT SELECT ON courses TO Assistant_employee;
GRANT Assistant_employee TO 'alice'@'localhost';
DROP ROLE Assistant_employee;
```

A **role** is a named collection of privileges that can be granted to (or revoked from) multiple users at once, simplifying access-control management.

8. Triggers

Concept

- A trigger is a statement executed automatically as a side-effect of a database modification.
- Defined by an **Event** (what causes the check), a **Condition** (must hold for execution), and an **Action** (what runs).
- Used for integrity constraints beyond plain SQL constraints, alerts, and automatic derived-data updates.

Syntax

```
CREATE TRIGGER trigger_name (AFTER | BEFORE) (INSERT |
UPDATE | DELETE) ON table_name
FOR EACH ROW
BEGIN
-- variable declarations
-- trigger code
END;
```

- **BEFORE / AFTER** — whether the trigger fires before or after the modification.
- **FOR EACH ROW** — the trigger body runs once per affected row.
- Inside the body, **NEW** refers to the new row values, **OLD** to the previous row values.

Example — validation trigger

```
DELIMITER //
CREATE TRIGGER CheckMarks
BEFORE INSERT ON Marks_sample
FOR EACH ROW
BEGIN
IF NEW.marks < 0 OR NEW.marks > 100 THEN SIGNAL SQLSTATE
'45000' SET MESSAGE_TEXT = 'Invalid marks';
END IF;
END;//
DELIMITER ;
```


Example — cascade-style update & audit backup

```
-- mimic ON UPDATE CASCADE
CREATE TRIGGER update_marks_on_cascade
BEFORE UPDATE ON Student_sample
FOR EACH ROW
BEGIN UPDATE Marks_sample SET SRN = NEW.SRN WHERE SRN =
OLD.SRN;
END;

-- backup deleted rows before removal
CREATE TRIGGER backup_marks_info
BEFORE DELETE ON Marks_sample
FOR EACH ROW
BEGIN
INSERT INTO Marks_history SELECT * FROM Marks_sample WHERE SRN =
OLD.SRN AND COURSE = OLD.COURSE;
END;
```

9. User-Defined Functions

Functions (and procedures) let developers store reusable “business logic” inside the database and invoke it from SQL statements.

Syntax

```
CREATE FUNCTION function_name (parameter_list) RETURNS data_type
BEGIN
-- statements
RETURN value;
END;
```

- A function always accepts parameters (0 or more) and returns a **single value**.
- Execution stops as soon as RETURN executes.
- **DETERMINISTIC** — same output for same input; functions are NON-DETERMINISTIC by default. • Delete with: DROP FUNCTION IF EXISTS function_name;

Example

```
CREATE FUNCTION Dept_size(deptno INT) RETURNS VARCHAR(7)
BEGIN
DECLARE No_of_ems INT; SELECT COUNT(*) INTO No_of_ems FROM
EMPLOYEE WHERE Dno = deptno;
IF No_of_ems > 3 THEN RETURN 'HUGE';
ELSEIF No_of_ems > 2 THEN RETURN 'LARGE';
ELSEIF No_of_ems > 1 THEN RETURN 'MEDIUM';
ELSE RETURN 'SMALL';
END IF;
END;

SELECT Dname, Dnumber, Dept_size(Dnumber) FROM department;
```

10. Stored Procedures

A stored procedure is a named, pre-compiled block of SQL statements stored in the database, invoked with CALL. Unlike functions it need not return a value and can have multiple parameters of different modes.

Syntax

```
DELIMITER && CREATE PROCEDURE procedure_name ([IN|OUT|INOUT] param
datatype, ...)
BEGIN
Declaration_section
Executable_section
END&&
DELIMITER ;
```

Parameter modes

Mode	Meaning
IN (default)	Caller passes a value in; procedure cannot change it for caller.
OUT	Procedure sets a value that is returned to the caller.
INOUT	Caller passes a value in, procedure may modify it, new value returned.

Examples

```
-- no parameters CREATE PROCEDURE
get_merit_student()
BEGIN
SELECT * FROM student WHERE marks > 70; SELECT
COUNT(student_code) AS Total_Student FROM student;
END;
CALL get_merit_student();

-- OUT parameter CREATE PROCEDURE
display_max_mark(OUT highestmark INT)
BEGIN SELECT MAX(marks) INTO highestmark FROM
student;
END;
CALL display_max_mark(@output); SELECT @output;
```

Drop a procedure with: `DROP PROCEDURE IF EXISTS procedure_name;` List procedures with `SHOW PROCEDURE STATUS.`

Note: Functions return exactly one value and can be used inline inside a `SELECT`; procedures may return multiple result sets / OUT values and are invoked with `CALL`, not inside `SELECT`.

11. Common Table Expressions (CTE)

WITH clause basics

A CTE defines a named temporary result set, available only within the query it is attached to. Improves readability versus deeply nested subqueries.

```
WITH cte_name AS (SELECT ...)
SELECT ... FROM cte_name;

-- multiple CTEs
WITH cte1 AS (...), cte2 AS (...)
SELECT ... FROM cte1 JOIN cte2 ...;

WITH max_work(max_hours) AS (SELECT MAX(Hours) FROM WORKS_ON)
SELECT Essn, Pno, Hours FROM WORKS_ON, max_work WHERE Hours = max_hours;
```

Recursive CTE

References itself; ideal for hierarchical data (e.g. employee–supervisor chains). Uses **WITH RECURSIVE** with an anchor member UNION ALL a recursive member.

```
WITH RECURSIVE cte_name(col_list) AS (
  SELECT ... -- anchor member (base case)
  UNION ALL SELECT ... FROM cte_name WHERE ... --
  recursive member
)
SELECT ... FROM cte_name;

-- organisation hierarchy from the CEO downward
WITH RECURSIVE orghierarchy AS ( SELECT ssn,
  fname, lname, super_ssn, 1 AS level FROM
  employee WHERE super_ssn IS NULL
  UNION ALL
  SELECT e.ssn, e.fname, e.lname, e.super_ssn, OH.level + 1
  FROM employee e JOIN orghierarchy OH ON e.super_ssn = OH.ssn
)
SELECT * FROM orghierarchy;
```

Note: Anchor member must not reference the CTE; the recursive member must include a stopping condition, otherwise the recursion never terminates.

CASE statement (often paired with CTE)

```
SELECT Fname, Salary,
CASE
WHEN Salary >= 55000 THEN 'High' WHEN Salary >= 40000 THEN 'Medium'
ELSE 'Low'
END AS Salary_Level
FROM employee;
```

If no WHEN condition is true and there is no ELSE, CASE returns NULL.

12. Window Functions

Concept

- Perform a calculation across a set of related rows **without collapsing** them into one output row — unlike GROUP BY / plain aggregates.
- Give both detailed row-level data *and* group-level aggregates/rankings in the same query.

```
SELECT column_list, window_function(col) OVER ([PARTITION BY partition_col] [ORDER BY
order_col])
AS new_column
FROM table_name;
```

- OVER()** tells SQL to keep every row and compute the value over a 'window'.
- PARTITION BY** splits rows into mini-groups (windows); the function is applied within each.

Types of window functions

Category	Functions
----------	-----------

Aggregate	SUM(), AVG(), COUNT(), MAX(), MIN() — used with OVER()
Ranking	ROW_NUMBER(), RANK(), DENSE_RANK()
Value	LAG(), LEAD(), FIRST_VALUE(), LAST_VALUE()

```
SELECT name, department, salary, AVG(salary) OVER (PARTITION BY department) AS
dept_avg_salary
FROM employees;
```

Ranking functions

Function	Behaviour on ties
RANK()	Same rank for ties; next rank is skipped (1,1,3,...)
DENSE_RANK()	Same rank for ties; next rank is consecutive (1,1,2,...)
ROW_NUMBER()	Unique sequential number always, no ties possible.

```
SELECT emp_name, department, salary, RANK() OVER (PARTITION BY department ORDER BY
salary DESC) AS rank_in_dept
FROM Employees;
```

Value functions — LAG / LEAD

LAG(col, offset, default) looks at a value from a *previous* row in the same partition; **LEAD(col, offset, default)** looks at a *future* row. Default offset = 1.

```
SELECT emp_name, department, salary, LAG(salary,1) OVER (PARTITION BY department ORDER
BY salary DESC) AS prev_salary
FROM Employees;
```

Practical patterns

```
-- Top-2 sales per employee
SELECT * FROM ( SELECT *, RANK() OVER (PARTITION BY EmpID ORDER BY Amount
DESC) AS SaleRank
FROM Sales
) t WHERE SaleRank <= 2;

-- running total
SELECT EmpID, Month, Amount, SUM(Amount) OVER (PARTITION BY EmpID
ORDER BY SaleDate) AS RunningTotal
FROM Sales;
```

13. Full-Text Search (MySQL)

Overview

- Searches large text columns (CHAR/VARCHAR/TEXT) using NLP-style relevance ranking, faster and smarter than LIKE/REGEXP.
- Requires a **FULLTEXT INDEX** on the target column(s); uses MATCH(col...) AGAINST('term').

```
CREATE TABLE posts (
  id INT AUTO_INCREMENT PRIMARY KEY, title
  VARCHAR(200), body TEXT ) ENGINE=InnoDB;
ALTER TABLE posts ADD FULLTEXT(title,
body);

SELECT * FROM posts
WHERE MATCH(title, body) AGAINST('database' IN NATURAL LANGUAGE MODE);
```

Search modes

Mode	Description
Natural Language (default)	Ranks by relevance; ignores stop-words and very short words.
Boolean Mode	Explicit operators for AND/OR/NOT-like, exact-phrase & wildcard search.

Relevance

Relevance is a floating-point score (higher = better match) based on term frequency, term rarity across rows, and document length. If no ORDER BY is used (and no JOIN), MySQL auto-sorts by relevance.

```
SELECT id, title, MATCH(title, body) AGAINST('database') AS relevance
FROM posts WHERE MATCH(title, body)
AGAINST('database')
ORDER BY relevance DESC;
```

Boolean mode operators

Operator	Meaning	Example
+	word must be present	+mysql
-	word must NOT be present	-database
""	exact phrase	"mysql tutorial"
*	suffix wildcard	data* → data, database
()	group terms	+(mysql database)

```
SELECT * FROM posts WHERE MATCH(title, body)
AGAINST('+mysql -oracle' IN BOOLEAN MODE);
```

— End of Unit 2 Notes —